Demonstrating Multi-Agent Collaboration via Agent-to-Agent and Model Context Protocols: An IT Incident Response Case Study

Vaibhav Tupe
Equinix
Redwood City, USA
vaibhav.k.tupe@gmail.com

Shrinath Thube
IBM
San Jose, USA
shrinaththube@gmail.com

2025 IEEE International Conference on Service-Oriented System Engineering (SOSE) | 979-8-3315-8911-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/SOSE67019.2025.00013

***Abstract*—This demonstration paper presents, a framework that showcases the integration of Google's Agent-to-Agent (A2A) Protocol and Model Context Protocol (MCP) within multi-agent AI systems. Using IT incident response as our demonstration example, we show how these protocols enable effective communication, context sharing, and problem-solving among AI agents with diverse capabilities. Our demonstration illustrates how Agent-to-Agent Protocol handles structured message passing and task delegation, while Model Context Protocol ensures consistent contextual understanding. Through simulated IT incidents of varying complexity, we demonstrate how this integrated approach improves response efficiency, reduces resolution time, and enhances automated remediation quality. This work provides insights into protocol interoperability challenges and presents a reference implementation adaptable for other multi-agent collaborative scenarios beyond IT operations.**

Keywords—A2A, MCP, agentic ai, Multi-Agent systems, autonomous agents, it incident response, protocol integration, automated problem solving

I. Introduction

Modern IT infrastructure demands increasingly sophisticated approaches to incident management. Traditional reactive methods are insufficient for handling the complexity and scale of contemporary computing environments. This demonstration presents a novel multi-agent system that addresses these challenges through advanced protocol integration and intelligent agent collaboration.

A. Motivation

The evolution of AI technologies has revealed limitations in agent communication approaches. Multi-agent systems struggle with context sharing, capability discovery, and incompatible messaging formats. Google's Agent-to-Agent (A2A) [1] Protocol and Anthropic's Model Context Protocol (MCP) [2] offer complementary approaches that can enable sophisticated multi-agent systems. Their integration could transform how agents collaborate across various domains by standardizing communication while preserving context.

Project Code - https://github.com/vaibhavtupe/multiagent-it-incident-response-system
Project Demo - https://youtu.be/2YCs6Axgifs

B. Related Work

Early multi-agent platforms such as JADE, IBM Aglets, and FIPA ACL [3][4][5] have demonstrated agent interoperability and structured communication, yet these typically lack unified context-sharing mechanisms essential for modern collaborative AI applications. Our demonstration is distinct in its integration of Google's Agent-to-Agent (A2A) Protocol and Anthropic's Model Context Protocol (MCP), enabling both structured message passing and consistent context propagation.

C. Innovative Approach

Our paper demonstrates protocol interoperability in action. The integration combines A2A Protocol's structured message passing and capability discovery with MCP's contextual awareness mechanisms. This creates a unified framework where agents can effectively communicate while maintaining shared understanding. Using IT incident response as our example domain, we show how this integration enables agents with different specializations to work together seamlessly, each leveraging their strengths while maintaining coherent collaboration.

D. Technical Significance

Our demonstration provides a practical reference architecture for protocol integration in multi-agent systems. We identify and address key challenges in cross-protocol communication, including capability representation, context transformation, and message routing. This work extends beyond IT incident response to any domain requiring collaborative problem-solving among diverse agents. The framework allows for future extensibility, supporting additional protocols while maintaining backward compatibility with existing agent ecosystems.

II. System Architecture

Our demonstration implements a layered architecture that integrates protocol technologies for effective multi-agent collaboration. The system consists of three main layers: the agent layer, the protocol integration layer, and the tool interaction layer. The agent layer houses specialized agents with

2642-6587/25/$31.00 ©2025 IEEE
DOI 10.1109/SOSE67019.2025.00013

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:35:25 UTC from IEEE Xplore. Restrictions apply.

distinct capabilities. The protocol integration layer combines A2A Protocol for structured communication and MCP for contextual understanding. The tool interaction layer provides access to specialized IT management tools through standardized interfaces. This architecture enables agents to collaborate across protocol boundaries while maintaining contextual coherence throughout the system.

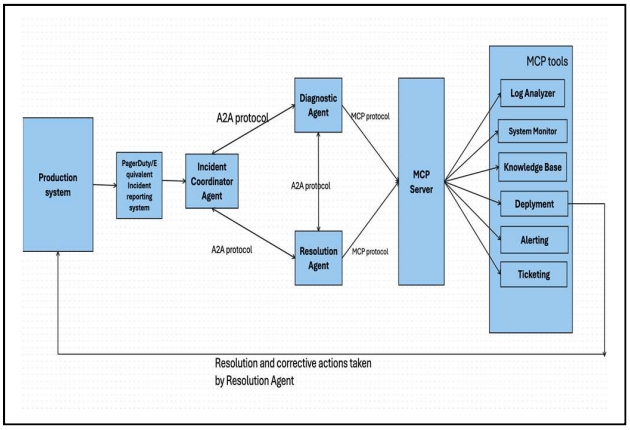


Fig. 1. Overall architecture showing how multiple agents use A2A and MCP protocols to build an incident response system.

A. Agents

The demonstration features three specialized agent types, each with distinct roles in the incident response process:

1) Incident Coordinator Agent

Orchestrates the overall response workflow, maintaining system-wide awareness and coordinating among other agents. This agent primarily uses A2A Protocol for task delegation and status tracking.

2) Diagnostic Agent

Analyzes system data to identify incident root causes using advanced analytical techniques. This agent leverages both protocols - A2A for communication and MCP for accessing diagnostic tools with proper context.

3) Resolution Agent

Implements solutions based on diagnostic findings. This agent relies heavily on MCP for secure access to deployment and configuration tools while using A2A to report resolution status to the coordinator.

B. Protocol Integration

Our approach demonstrates seamless integration between two complementary protocols:

1) Agent-to-Agent (A2A) Protocol

Provides the messaging framework that enables agents to discover capabilities, negotiate tasks, and coordinate activities. We implement key A2A features including structured message formats, capability advertisement, and task delegation patterns.

2) Model Context Protocol (MCP)

Supplies the contextual awareness layer that ensures consistent understanding across agents and tools. Our implementation focuses on how MCP maintains context integrity during complex multi-step operations across different agent boundaries. Fig. 2. Shows MCP architecture.

The integration layer translates between protocol formats when necessary, ensuring that context established in one protocol remains coherent when accessed through the other. This bidirectional integration enables new patterns of agent collaboration that would be impossible with either protocol alone.

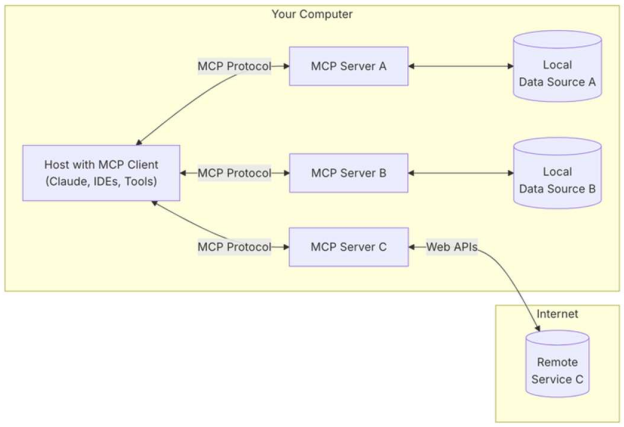


Fig. 2. MCP general architecture [2]

C. MCP Server and MCP Tools

The demonstration includes an MCP server that manages context across multiple specialized tools:

- Log Analyzer: Provides advanced log processing capabilities with contextual awareness of incident parameters.
- System Monitor: Delivers real-time performance metrics within the established incident context.
- Knowledge Base: Supplies relevant historical information filtered by current incident context.
- Deployment System: Implements configuration changes with awareness of incident scope and impact.
- Ticketing System: Maintains formal incident records with appropriate contextual annotations.

These tools communicate with agents through the MCP server, which ensures that all interactions maintain appropriate context. The MCP server also facilitates secure capability discovery, allowing agents to find and utilize the most appropriate tools for specific incident contexts.

III. Implementation Details

A. Project Structure

The project follows a standard Python package structure designed to provide modularity, extensibility, and clear separation of concerns. The root directory contains essential project configuration files, including setup.py for package

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:35:25 UTC from IEEE Xplore. Restrictions apply.

installation, requirements.txt for dependency management, and run_demo.py as the primary demonstration entry point.

The it_incident_response/ core package directory is organized into specialized subdirectories:

- protocols/: Contains implementations of A2A and MCP communication protocols
- agents/: Defines the intelligent agent implementations
- tools/: Houses the MCP-enabled tool integrations
- models/: Defines data models and structures
- simulation/: Provides simulation and testing utilities
- system.py: Orchestrates the overall system integration

The tests/ directory contains comprehensive unit and integration tests to ensure system reliability and functionality. Project Repository: The complete source code is available on GitHub at: https://github.com/vaibhavtupe/multiagent-it-incident-response-system

```
it_incident_response/
├── README.md
├── requirements.txt
├── setup.py
├── run_demo.py
├── it_incident_response/
│   ├── __init__.py
│   ├── protocols/
│   ├── agents/
│   ├── tools/
│   ├── models/
│   ├── simulation/
│   └── system.py
└── tests/
```

B. Workflow Demonstration

The run_demo.py script serves as the primary demonstration mechanism, orchestrating a comprehensive end-to-end IT incident response workflow. To execute the demonstration, one can follow the standard installation and run procedures:

1) System Requirements

- Python 3.8+
- Dependencies listed in requirements.txt

2) Installation

```
# bash
pip install -e .
```

3) Running the Demonstration

```
# bash
python run_demo.py
```

The run_demo.py script demonstrates the complete incident response workflow:

1. Incident Detection
2. Diagnostic Analysis
3. Root Cause Identification
4. Solution Generation
5. Implementation
6. Verification and Closure.

IV. Experimental Results

A. Experimental Methodology

The experimental validation employs a comprehensive testing approach that demonstrates the multi-agent system's capabilities through a carefully constructed database connectivity incident simulation. The methodology focuses on validating the system's ability to detect, diagnose, and resolve complex IT infrastructure challenges autonomously.

1) Simulated Incident Scenario

- Type: Database Connectivity Degradation
- Affected Systems:
  - Application Server (app-server-01)
  - Database Server (db-server-02)
- Simulated Anomalies:
  - Network latency spikes
  - Excessive database connection times
  - Intermittent performance degradation

2) Performance Metrics and Analysis

TABLE I. Performance Metrics and Observations

Metric	Threshold	Observed Values	Interpretation
Root Cause Detection Accuracy	-	85%	Overall detection performance
Database Connection Time Analysis	500 ms	1282.76ms, 1062.62ms, 949.72ms, 1943.38ms, 1461.99ms	Higher than threshold
Network Latency Measurements	50 ms	194.63ms, 71.32ms, 137.35ms, 131.14ms, 262.50ms	Higher than threshold

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:35:25 UTC from IEEE Xplore. Restrictions apply.

3) Resolution and Verification

TABLE II. RESOLUTION STRATERGIES

Category	Strategies
Dynamic Resource Allocation	Increased resources for affected services by adaptive scaling based on diagnostic insights
Monitoring Enhancements	Additional logging mechanisms and comprehensive error pattern tracking
System Configuration Optimization	Validated deployment configurations and reviewed recent system changes

TABLE III. VARIFICATION METRICS

Category	Details
Connection Stability Test	Validated network connectivity & connection reliability
Load Performance Test	Simulated peak traffic and assessed system responsiveness
Error Rate Monitoring	Tracked error frequencies
Resource Utilization Check	Monitored CPU, memory, and network resources

B. Experimental Report





V. CONCLUSION

This demonstration paper has presented a practical approach to integrating Google's Agent-to-Agent Protocol with Anthropic's Model Context Protocol in multi-agent systems. Our implementation shows how these complementary protocols can work together to enable more effective agent collaboration while maintaining contextual coherence. The IT incident response domain provides clear evidence of the benefits of protocol interoperability, including improved coordination, consistent context management, and more flexible agent interactions. Beyond IT incident response, the proposed integration framework can be extended to other domains requiring robust multi-agent collaboration, such as supply chain management, security automation, and healthcare coordination. The modular architecture and standardized protocol interfaces facilitate adaptation to new scenarios with minimal changes. Ongoing work will focus on supporting additional protocols and further validating extensibility across diverse application areas.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:35:25 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Google Research, "Google A2A Protocol Specification," 2023. [Online]. Available: https://google.github.io/A2A/ [Accessed: Apr. 21, 2025].

[2] Anthropic, "Anthropic Model Context Protocol Documentation," 2024. [Online]. Available: https://modelcontextprotocol.io/introduction [Accessed: Apr. 21, 2025].

[3] JADE, https://jade.tilab.com/

[4] IBM Aglets, https://aglets.sourceforge.net/

[5] FIPA, ACL, http://www.fipa.org/specs/fipa00061/SC00061G.html

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:35:25 UTC from IEEE Xplore. Restrictions apply.